

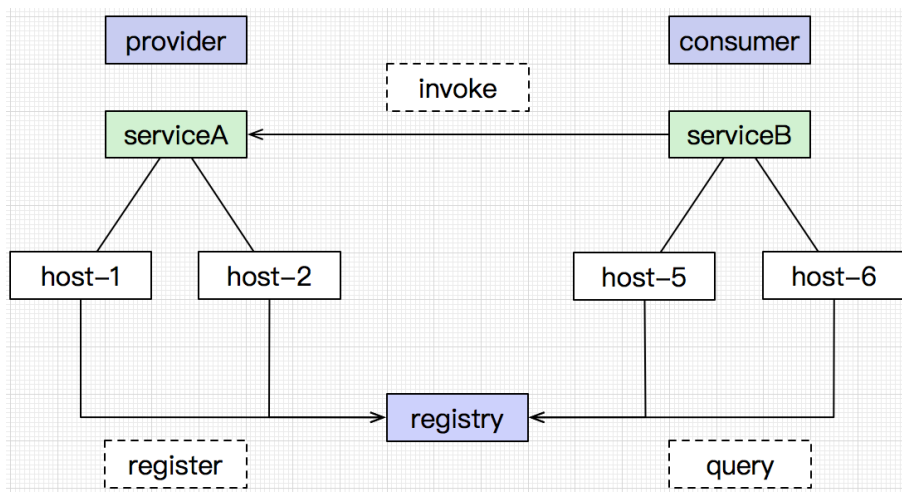
RPC 的含义与架构

RPC 的全称为远程过程调用，实现多个服务之间的调用依赖，在系统架构中有非常重要的作用。

RPC 的角色：

- 提供者 provider，提供 1 个或多个服务
- 消费者 consumer，使用 1 个或多个服务
- 注册中心 registry，记录全部服务的提供者、消费者

RPC 的核心架构图：



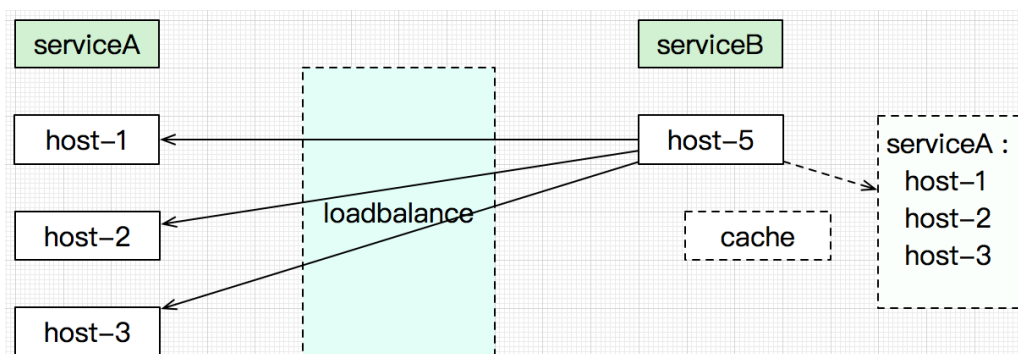
RPC 的功能细节比较复杂，这里着重分析 RPC 的负载均衡。

RPC 的负载均衡

RPC 的负载均衡集中体现在消费者调用服务、注册中心缓存服务提供者。

- 消费者使用服务

分布式环境，某个服务的提供者可以有多个，消费者使用负载均衡算法选择提供者，然后发起 RPC 调用。负载均衡算法，在前文已经讲解。RPC 常用轮询法、随机法。



- 注册中心缓存服务提供者

提供者定期上报本机状态到注册中心，注册中心把服务与提供者的映射关系存入缓存，提供高效查询。缓存提供者可以用类名，也可以用方法名。

假设提供者代码如下：

```
package com.test1;

public interface PhotoService {
    public String queryPhoto(String photoUrl, int type);
}
```

类名缓存法：缓存的 key 为类名

```
cache key=com.test1.PhotoService
cache value=192.168.1.100, 192.168.1.101, 192.168.1.102
```

方法名缓存法：缓存的 key 为类名+方法名

```
cache key=com.test1.PhotoService#queryPhoto(String, int)
cache value=192.168.1.105, 192.168.1.106, 192.168.1.107
```

对比可知，类名缓存法与方法名缓存法的差异主要在于缓存 key 的规则。

方法名缓存法，缓存 key 细化到方法名，粒度更细，对于多个机器使用不同服务版本更合适。

因为不同服务版本可能包含不同的方法。

比如，192.168.1.105 的服务包含方法 queryPhoto，但是 192.168.1.109 的服务不包含方法 queryPhoto，则 PhotoService#queryPhoto 的缓存 value 包含 192.168.1.105、不包含 192.168.1.109。

引申：缓存 key 还可以包含服务的版本号，实现灰度中间件；也可以在缓存 value 增加 metadata 功能，填入版本号等扩展数据。

注册中心有多种实现方式，能够保存 key-value 结构数据就行，比如 zookeeper、redis、consul、mysql 等。

Dubbo 实现负载均衡

Dubbo 属于优秀的 RPC 中间件，包含组件化设计、分层架构、支持多种序列化等优点。

Dubbo 负载均衡的核心类：

- RoundRobinLoadBalance
- RandomLoadBalance
- ConsistentHashLoadBalance
- LeastActiveLoadBalance

Dubbo 负载均衡使用多种算法，包括权重轮询法、权重随机法、一致性哈希法等。

Dubbo 权重轮询法的主要代码：

```
private final ConcurrentMap<String, AtomicPositiveInteger> sequences = new
ConcurrentHashMap<String, AtomicPositiveInteger>();

private final ConcurrentMap<String, AtomicPositiveInteger> weightSequences = new
ConcurrentHashMap<String, AtomicPositiveInteger>();

protected <T> Invoker<T> doSelect(List<Invoker<T>> invokers, URL url, Invocation invocation) {
    String key = invokers.get(0).getUrl().getServiceKey() + "." + invocation.getMethodName();
```

```

int length = invokers.size(); // 总个数
int maxWeight = 0; // 最大权重
int minWeight = Integer.MAX_VALUE; // 最小权重
for (int i = 0; i < length; i++) {
    int weight = getWeight(invokers.get(i), invocation);
    maxWeight = Math.max(maxWeight, weight); // 累计最大权重
    minWeight = Math.min(minWeight, weight); // 累计最小权重
}
if (maxWeight > 0 && minWeight < maxWeight) { // 权重不一样
    AtomicPositiveInteger weightSequence = weightSequences.get(key);
    if (weightSequence == null) {
        weightSequences.putIfAbsent(key, new AtomicPositiveInteger());
        weightSequence = weightSequences.get(key);
    }
    int currentWeight = weightSequence.getAndIncrement() % maxWeight;
    List<Invoker<T>> weightInvokers = new ArrayList<Invoker<T>>();
    for (Invoker<T> invoker : invokers) { // 筛选权重大于当前权重基数的 Invoker
        if (getWeight(invoker, invocation) > currentWeight) {
            weightInvokers.add(invoker);
        }
    }

    int weightLength = weightInvokers.size();
    if (weightLength == 1) {
        return weightInvokers.get(0);
    } else if (weightLength > 1) {
        invokers = weightInvokers;
        length = invokers.size();
    }
}
AtomicPositiveInteger sequence = sequences.get(key);
if (sequence == null) {
    sequences.putIfAbsent(key, new AtomicPositiveInteger());
    sequence = sequences.get(key);
}
// 取模轮循
return invokers.get(sequence.getAndIncrement() % length);
}

```

Dubbo 权重随机法的主要代码:

```

private final Random random = new Random();

protected <T> Invoker<T> doSelect(List<Invoker<T>> invokers, URL url, Invocation invocation) {
    int length = invokers.size(); // 总个数
    int totalWeight = 0; // 总权重
    boolean sameWeight = true; // 权重是否都一样
    for (int i = 0; i < length; i++) {
        int weight = getWeight(invokers.get(i), invocation);
    }
}

```

```

        totalWeight += weight; // 累计总权重
        if (sameWeight && i > 0
            && weight != getWeight(invokers.get(i - 1), invocation)) {
            sameWeight = false; // 计算所有权重是否一样
        }
    }

    if (totalWeight > 0 && ! sameWeight) {
        // 如果权重不相同且权重大于 0 则按总权重数随机
        int offset = random.nextInt(totalWeight);
        // 并确定随机值落在哪个片断上
        for (int i = 0; i < length; i++) {
            offset -= getWeight(invokers.get(i), invocation);
            if (offset < 0) {
                return invokers.get(i);
            }
        }
    }
    // 如果权重相同或权重为 0 则均等随机
    return invokers.get(random.nextInt(length));
}

```

Spring RESTful 实现负载均衡

Spring 可以集成多种 RPC 方式，这里分析 RESTful 方式，使用 spring-cloud-starter-zookeeper-discovery 的 4.0.1 版本。

RESTful 负载均衡的核心类：

- ReactiveLoadBalancer
- RoundRobinLoadBalancer
- RandomLoadBalancer

RESTful 负载均衡使用多种算法，包括轮询法、随机法等。

RESTful 轮询法的主要代码：

```

final AtomicInteger position;

private Response<ServiceInstance> getInstanceResponse(List<ServiceInstance> instances) {
    if (instances.isEmpty()) {
        if (log.isWarnEnabled()) {
            log.warn("No servers available for service: " + serviceId);
        }
        return new EmptyResponse();
    }

    // Do not move position when there is only 1 instance, especially some suppliers
    // have already filtered instances
    if (instances.size() == 1) {
        return new DefaultResponse(instances.get(0));
    }
}

```

```
// Ignore the sign bit, this allows pos to loop sequentially from 0 to
// Integer.MAX_VALUE
int pos = this.position.incrementAndGet() & Integer.MAX_VALUE;

ServiceInstance instance = instances.get(pos % instances.size());

return new DefaultResponse(instance);
}
```

RESTful 随机法的主要代码：

```
private Response<ServiceInstance> getInstanceResponse(List<ServiceInstance> instances) {
    if (instances.isEmpty()) {
        if (log.isWarnEnabled()) {
            log.warn("No servers available for service: " + serviceId);
        }
        return new EmptyResponse();
    }
    int index = ThreadLocalRandom.current().nextInt(instances.size());

    ServiceInstance instance = instances.get(index);

    return new DefaultResponse(instance);
}
```

小结

综合 Dubbo、Spring 实现负载均衡的方式，轮询法、随机法使用广泛，因其算法简单且高效。
架构思维：大道至简，复杂的系统依赖简单的组件。